

LAB05: Tiny Transformers on PULP

Davide Nadalini – d.nadalini@unibo.it
Luca Bompani – luca.bompani5@unibo.it
Alberto Dequino – alberto.dequino@unibo.it
Francesco Conti – f.conti@unibo.it

Objective of the Class

Intro: Tiny Transformers on PULP

Tasks: execution of a TinyLlama on PULP

- Attention block
- KV cache
- Autoregressive text generation
- Positional encoding
- Tiny story generation with TinyLlama

Deadline:

Nov 7st 2025

Programming Language: C

Lab duration: 3h

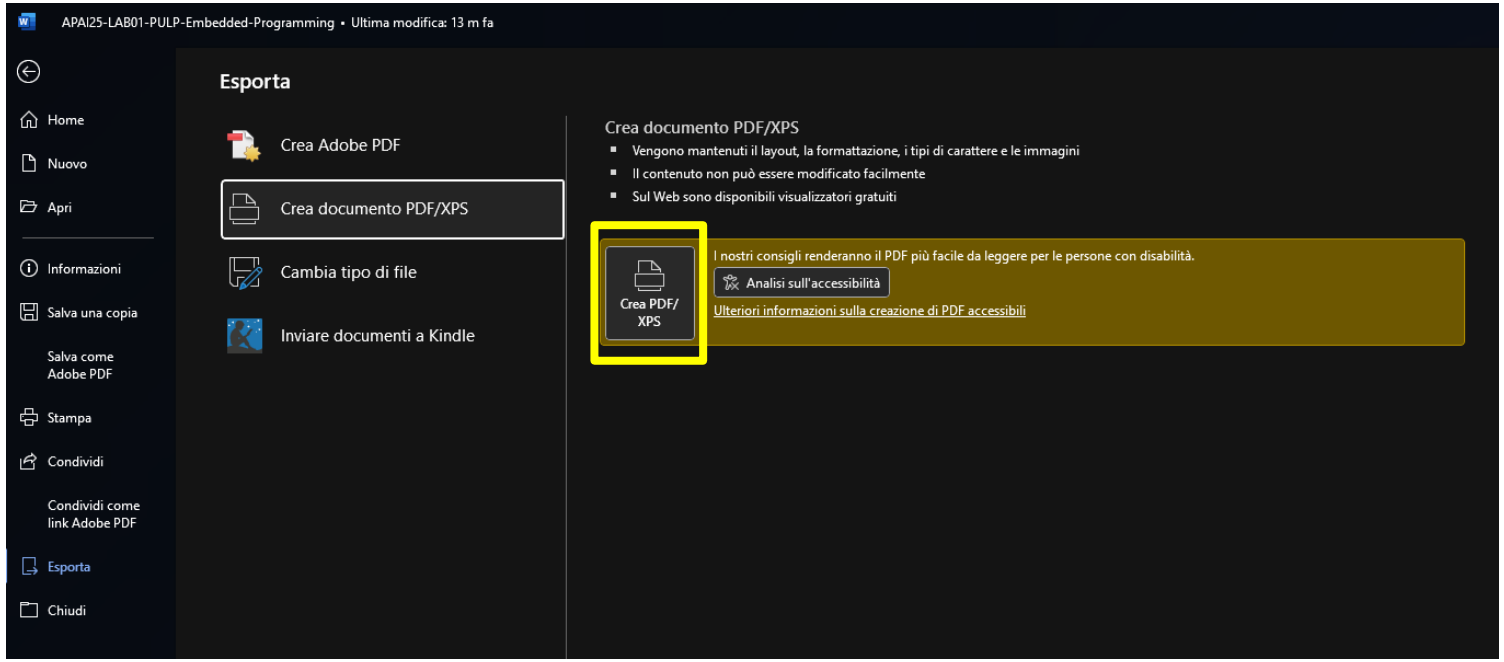
The class is meant to be interactive: coding together and on your own!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

How to deliver the Assignment

You will deliver ONLY the PDF assignment, no code

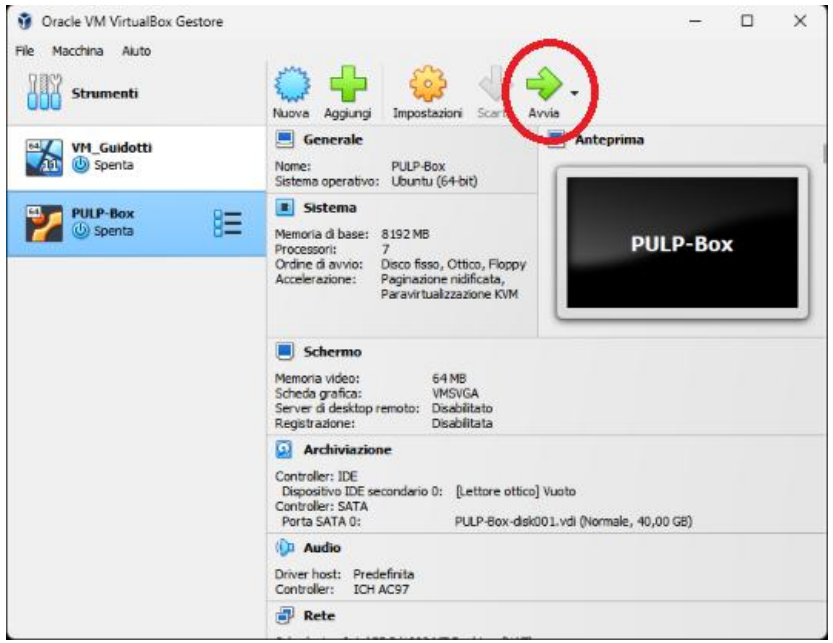
- Download the assignment file from Virtuale.
- Fill the results required by the assignment.
- Export to pdf format.
- Rename the file to: LAB<number_of_the_lesson>_APAI_<your_name>.pdf
- Use Virtuale platform to load ONLY your .pdf file

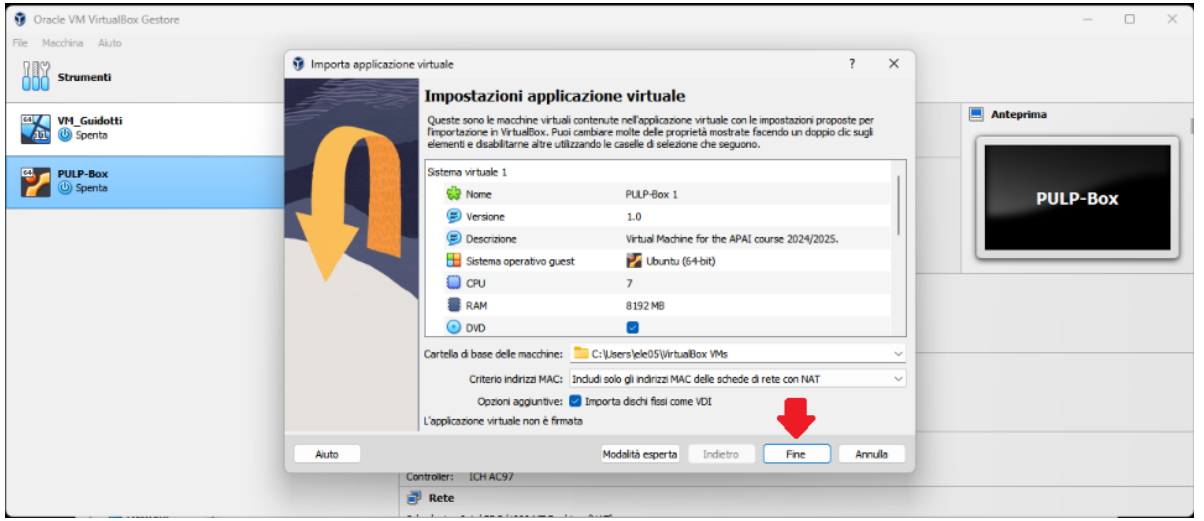


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Opening the VM and VSCode

1. On the lab's PCs, open the file explorer and go to This PC, C:/VM_APAI/
2. Double click on PULP-box.ova
3. VirtualBox opens, just click on "Fine"
4. Wait for the VM to be imported
5. Open the VM with "Avvia"





Password is 'pulp'

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

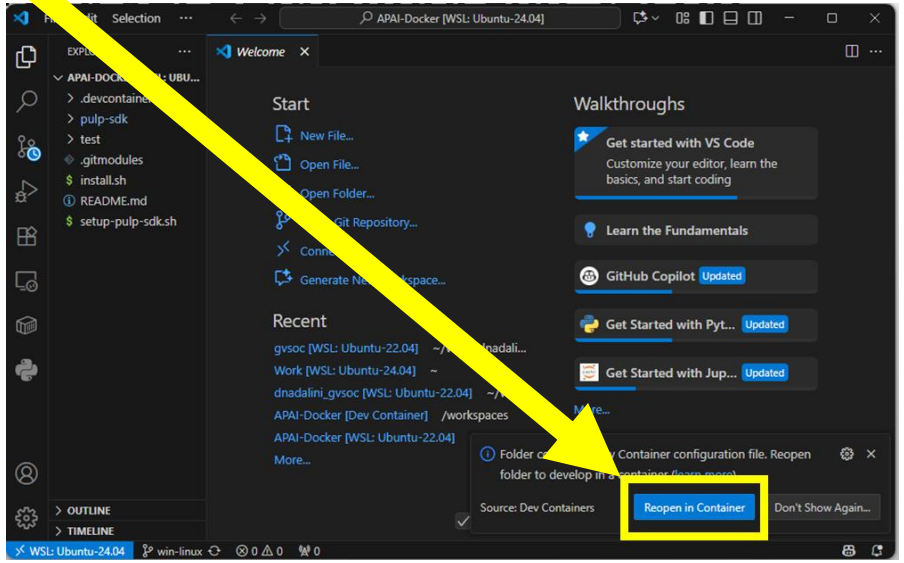
Opening the Docker with VSCode

1. Open a terminal (right click – open a new terminal)
2. From the terminal, open VSCode in the folder of the Docker

```
$ cd APAI-Docker
$ code .
```

1. Reopen the APAI-Docker folder in VSCode (click on “Reopen in container”)

1. Now you can use the integrated terminal (open with CTRL+J) to run your applications!



IMPORTANT: every time you open a new terminal to work on PULP, launch

```
$ source setup-pulp-sdk.sh
```


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Getting Started:

IMPORTANT: activate the pulp-sdk module file every time a new shell is open.

```
$ source setup-pulp-sdk.sh
```

HOW TO RUN THE CODE:

```
$ git clone https://github.com/EEESlab/APAI25-LAB05-TinyTransformers
$ cd APAI25-LAB05-TinyTransformers
$ cd <folder_you_want>
$ make clean all run
```

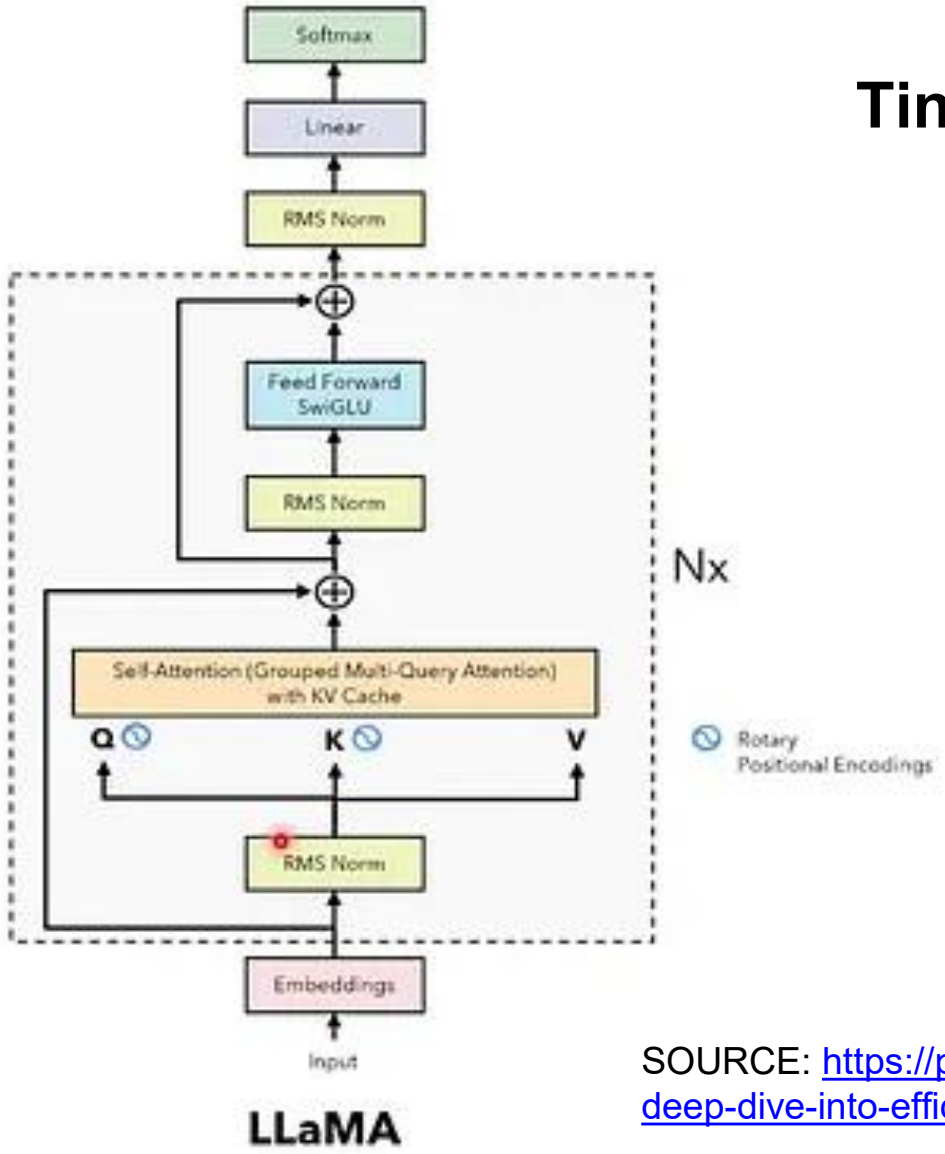

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

TinyLlama

TinyLlama



TinyLlama: tiny language model based on a Transformer architecture

Decoder-only model:

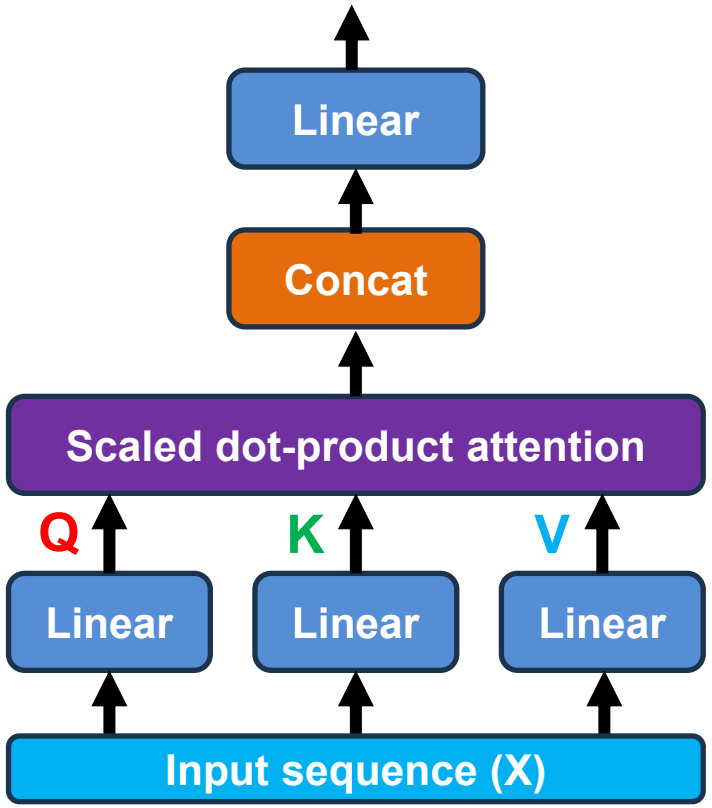
- No Encoder/Decoder, single architecture
- Autoregressive: outputs and inputs are used for text generation

SOURCE: https://pub.towardsai.net/llama-architecture-a-deep-dive-into-efficiency-and-mathematics-9c95c0e4bf8b

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Attention Block: Optimization

Core: Attention Block



Given an (encoded) input sequence X, the attention block computes:

Queries: $Q = XW^Q$

Keys: $K = XW^K$

Values: $V = XW^K$

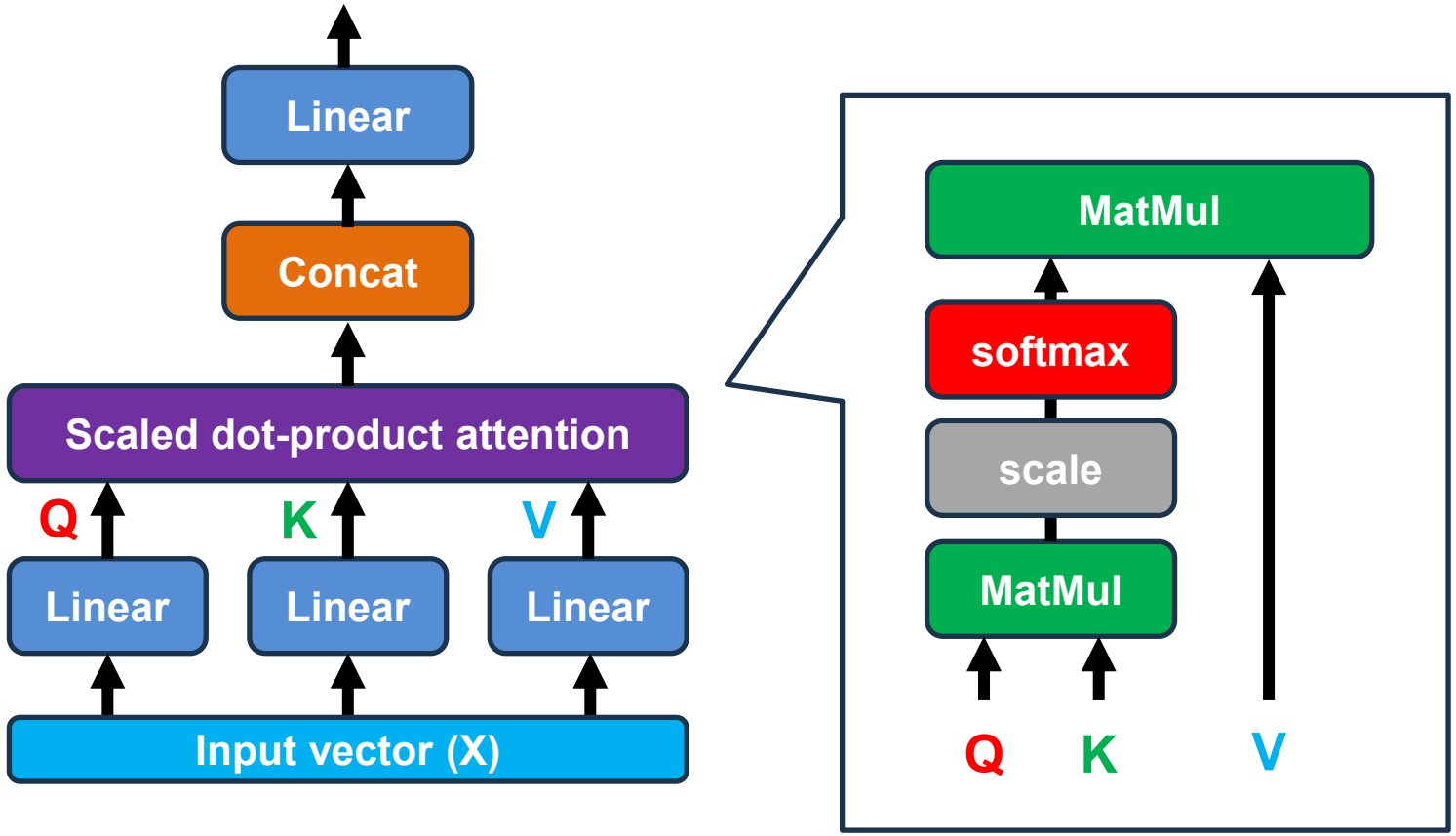
Which are MatMuls!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Attention Block: Optimization

Core: Attention Block

$$Attention(Q, K, V) = softmax\left(\frac{QK^t}{\sqrt{d_k}}\right)V$$

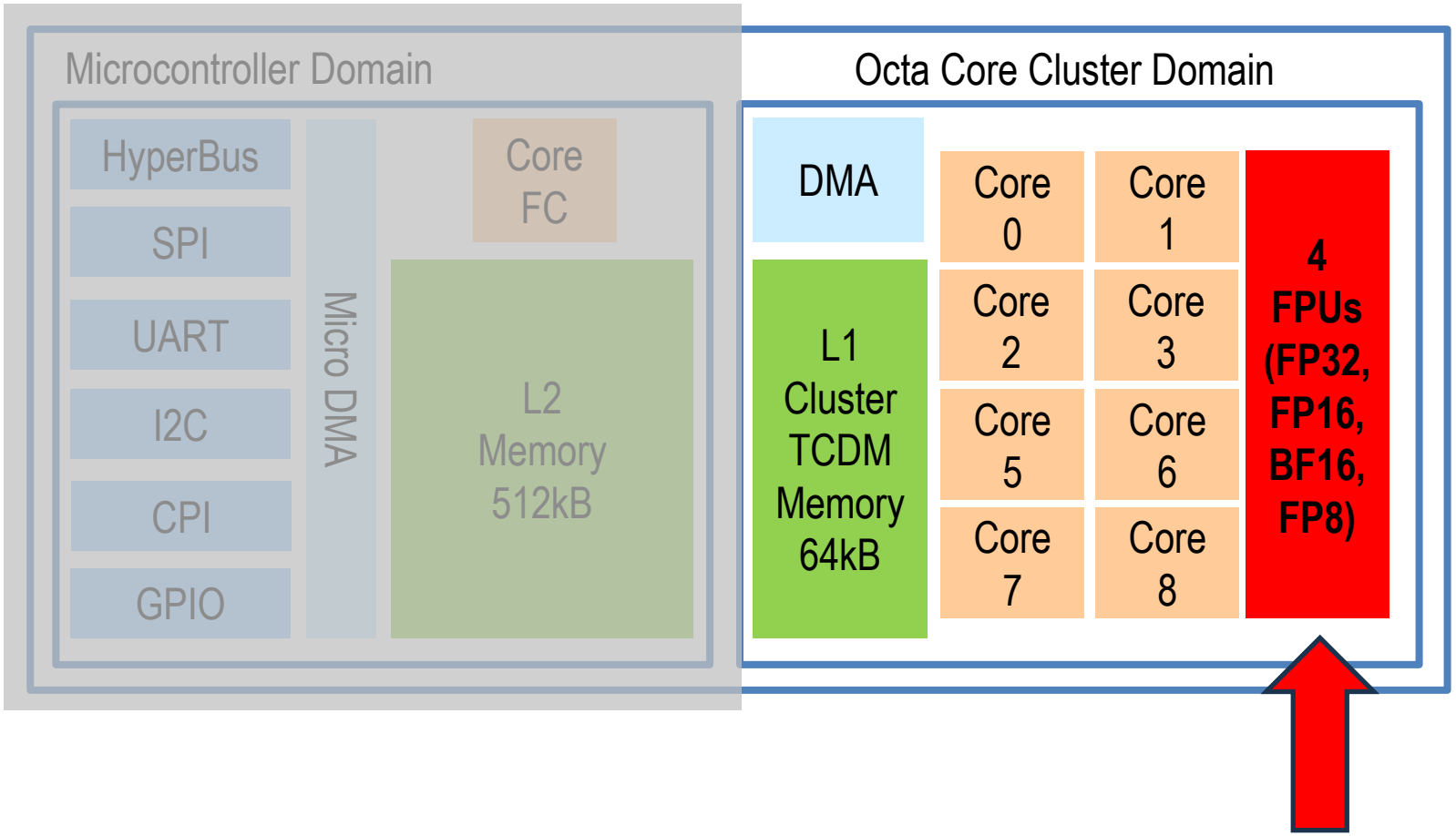


The attention block contains the softmax operation, which has exponentials in it

We use FP16 instead of INT8/INT16

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

PULP FPUs



However, the PULP Cluster has a mixed-precision floating-point unit (FPU) which can support both FP32, FP16, BF16 and more!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Mini-Floats



Floating-Point 32 (FP32)
IEEE 754 standard (see lecture on Quantization)

Mini-Floats: reduce precision for SIMD and 0.5x memoy usage!!



IEEE FP16 – efficient for inference, but not for training



BFloat16 – good tradeoff for inference & training!!

In some cases, even 8-bit floating point can be used!!
See, for example: https://arxiv.org/abs/2209.05433

Using vectorized mini-floats in PULP-SDK

PULP features mini-float data types, as FP16 or BFloat16 as native types in the compiler (e.g., BFloat16 as “float16alt”)

```
typedef float16alt fp16;                                    // FP16 format (float16 or float16alt)
typedef fp16 v2f16 __attribute__((vector_size (4)));        // Vectorized fp16 for SIMD
```

Vectorized FP16 data can be defined as “vectors of 2 elements” for SIMD instructions to exploit the mixed-precision FPU!

Using vectorized mini-floats in PULP-SDK (cont'd)

```
void vm_T_SIMD (void * void_args)
{
    struct matMul_args_fp16 * args = (struct matMul_args_fp16 *) void_args;
    fp16 * A = args->A;     // In this example, the Output Gradient
    fp16 * B = args->B;     // In this example, the Weight matrix
    fp16 * C = args->C;     // In this example, the Input Gradient
    uint32_t M = args->M;
    uint32_t K = args->K;

    for (int j = 0; j < M; j++)
    {
        v2f16 temp = (v2f16) {0, 0};
        for (int k = 0; k < K; k+=2)
        {
            // Load both adjacent elements for A and B
            v2f16 Av = *((v2f16*) &A[k]);
            v2f16 Bv = *((v2f16*) &B[j*K + k]);
            temp += Av * Bv;
        }
        C[j] = temp[0] + temp[1];
    }
}
```

Initialize a FP16 vector of 2 elements

Load FP16 vectors

Multiply FP16 vectors with SIMD

Vector types (v2f16) can also be used to execute SIMD instruction in a Vector Matrix multiplication, for example!



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DEI – Università di Bologna

www.unibo.it